Muhammed Gölcük 21802551

Metehan Küçük 21802120

EEE-485 Fall 2023 Final Project Report

Application of Deep Reinforcement Learning & Reinforcement Learning for Breakout Game

1.0 Introduction

This research project is centered around the exploration and implementation of two pivotal reinforcement learning (RL) algorithms, namely Q-Learning and SARSA (State-Action-Reward-State-Action), and one deep reinforcement learning (DRL) algorithm, the Deep Q Network (DQN), within the context of a simulated environment. The chosen environment for this study is the classic Atari Breakout game, a well-acknowledged testbed in the realm of RL owing to its intricate dynamics and the rich learning opportunities it presents [1].

The choice of the Atari Breakout game as our testing ground is deliberate. Its status as a classic in the domain of RL makes it an ideal platform for analyzing and honing the capabilities of RL algorithms. The game's environment offers a complex yet controllable landscape for training autonomous agents, making it a quintessential example for this study. Our research will delve into the nuances of SARSA, Q-Learning and DQN, comparing their methodologies, challenges, and unique attributes within the context of this game [2].

Furthermore, this report aims to present a thorough evaluation of these algorithms. We will discuss performance metrics, which are crucial for assessing the effectiveness and efficiency of the algorithms in achieving the desired learning outcomes. Additionally, this report will provide a detailed breakdown of the contributions made by each team member, ensuring a transparent and comprehensive understanding of the collaborative efforts involved in this project.

In summary, this introduction sets the stage for a detailed examination of SARSA, Q-Learning and DQNalgorithms within the Atari Breakout game environment. Our goal is to not only implement these algorithms successfully but also to extract valuable insights about their performance and applicability in complex RL scenarios.

2.0 Problem and Dataset Description

This section encompasses a comprehensive exploration of the problem definition and dataset description, subdivided into three elucidating subsections.

2.1 Problem Definition

In the game of Breakout, players engage in a stimulating challenge where they maneuver a paddle to propel a ball towards an intricately structured wall of bricks. The primary objective is to skillfully dismantle these bricks to accumulate coveted points. The inherent dynamism of this gaming scenario presents a distinctive challenge for a Reinforcement Learning (RL) agent. The agent is tasked with developing and refining a strategic approach that maximizes scoring efficiency amid ever-changing conditions.

Our strategic approach to solving this intricate problem centers around model-free algorithms. Despite Breakout being a deterministic game with a deterministic Markov Decision Process (MDP), its vast state space necessitates the utilization of model-free algorithms, such as SARSA ,Q-learning and DQN, as opposed to temporal difference learning.

2.2 Dataset Description

In the conventional realm of machine learning, a predefined dataset serves as a fundamental prerequisite. However, the landscape of RL introduces a paradigm shift in the concept of a dataset. RL

agents dynamically interact with their environment, generating data through these interactions. This evolving data encompasses diverse states of the environment, the actions undertaken by the agent, and the ensuing rewards or penalties. In the context of Breakout, this dynamic "dataset" is continuously evolving and comprises the following elements:

2.2.1 State Space Representation: The state space in Breakout intricately includes the position and velocity of the ball, the paddle's position, and the configuration of the bricks. This multifaceted representation forms the foundation for comprehending the environment's dynamics.

2.2.2 Action Space: The agent is endowed with a set of actions, typically involving moving the paddle left, right, or maintaining a stationary position. These actions play a pivotal role in shaping the agent's responses to the evolving game dynamics.

2.2.3 Rewards System: Defined by the game's scoring rules, the rewards system is integral to the learning process. Breaking each brick contributes to the cumulative score, influencing the agent's strategic decision-making.

Armed with this rich information in our dynamically evolving dataset, we leverage Bellman equations as the cornerstone for SARSA, Q-learning and by extension DQN. Two crucial Bellman Equations guide our algorithms:

Bellman Equation for Value State Function:

$$V^{\pi}(s) = \sum_{a \in A} \pi(a|s) \sum_{s',r} P(s,a)[r + \gamma V^{\pi}(s')]$$

Bellman Equation for Action-Value Function:

$$Q^{\pi}(s,a) = \sum_{s',r} P(s,a)[r + \gamma \sum_{a'} P(s')\, Q^{\pi}(s',a')]$$

These equations are important in many ways such as the action value function allowing us to handle high-dimensional action spaces, off-policy learning, which are the Q learning and DQN, and the State Value function used in on-policy learning where we use SARSA. These are helpful in these algorithms since the value state function calculates the expected return under a certain policy without specifying initial action. While Action Value calculates the expected return with action *a* while in states [1], [2].

For off-policy RL algorithms, such as Q-learning and DQN, a "replay buffer" is employed. This mechanism stores past experiences (state, action, reward, next state) and uses them to update the agent's policy. This buffer allows the agent to learn from a more diverse set of experiences, thereby improving learning efficiency and stability.

2.3 Preprocessing

Preprocessing involves translating the game's pixel data into meaningful state representations. Training involves iterative gameplay, with discount factor and learning rate tuning to optimize learning.

3.0 Review of Machine Learning Methods

In this section, we delve into an in-depth review of the machine learning models that have been implemented and assessed in the context of our project.

3.1 SARSA Implementation

Muhammed Gölcük 21802551

Metehan Küçük 21802120

SARSA, an on-policy Reinforcement Learning (RL) algorithm, was chosen primarily due to its effectiveness in environments where exploration is key. In adapting SARSA for the Breakout game, we emphasized the learning of state-action pairs, leveraging the game's reward system, which revolves around brick destruction and ball control. The core algorithmic equation for SARSA is as follows:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$$

where α is the learning rate, γ is the discount factor, and r_{t+1} the reward. Although similar in structure to Q-learning and temporal difference learning, SARSA differs significantly in its implementation, as will be elucidated in subsection 3.3.

For SARSA, the Q-table is iteratively updated, with a keen focus on tuning the learning rate α and discount factor γ . The state space is discretized to represent different positions and movements of the paddle and ball [2].

3.2 Q-learning Implementation

Q-learning, a model-free off-policy RL algorithm, focuses on identifying the optimal action regardless of the current policy. In our application to Breakout, the goal was to maximize future rewards. The update rule for Q-learning is as follows:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_{t+1} + \gamma max_a Q(s_{t+1}, a) - Q(s_t, a_t))$$

Q-learning, being an off-policy algorithm, utilizes a replay buffer which stores state, action, and reward pairs for the determination of the maximum Q in policy development [1].

3.3 Comparison of SARSA and Q-learning

Comparing these algorithms in the Breakout context allows us to assess their efficiency and effectiveness in different game scenarios, particularly in balancing exploration with exploitation challenges.

Q-learning's implementation also involves a Q-table, with specific attention to off-policy learning dynamics. Exploration strategies, such as epsilon-greedy, can be employed to enhance the learning process.

SARSA is an on-policy algorithm that doesn't learn from its past experiences. Therefore, weights exploration over exploration, unlike Q learning, where the replay buffer stores the information for future policy determination, which is an off-policy algorithm.

Furthermore, the term $max_a Q(s_{t+1}, a)$ often causes an overestimation problem since $max_a Q(s_{t+1}, a)$ selects the maximum state action reward pair, which often comes with sensory noise ε. Therefore, when such a case occurs, convergence time increases and creates a variance between each run of episodes.

As outlined in the problem definition 2.1, the state space of the Breakout game is extensive, necessitating the use of model-free learning instead of model-based learning, as calculating T*(s, a s'), P(s a s') is* impractical. Therefore, we avoid temporal learning that relies on these Markov Decision Process (MDP) models [2].

3.4 DQN Implementation [1], [2]

Deep Q-Network (DQN) represents a significant advancement in the realm of RL, combining Q-learning with the power of deep neural networks. DQN extends the capabilities of traditional Q-learning by employing neural networks to approximate the Q-value function. This approach enables the handling of environments with high-dimensional state spaces, such as the Breakout game.

Muhammed Gölcük 21802551

Metehan Küçük 21802120

In our DQN implementation, the neural network serves as a function approximator for the Q-value of each action in a given state. The key innovation of DQN lies in its ability to generalize the action-value function across similar states, thereby enhancing learning efficiency and robustness.

DQN also employs techniques like experience replay and fixed Q-targets to address issues of data correlation and non-stationarity, common challenges in deep learning-based RL. The experience replay mechanism stores the agent's experiences in a replay buffer, similar to Q-learning, but with a more sophisticated approach to sampling and utilizing these experiences.

In the implementation of the algorithm an CNN class created, as a loss function we used Huber loss and in contrast to the Q learning and SARSA we created a DeepQ agent which utilizes the CNN class and other required functionalities, also epsilon greedy algorithm was tested. Environment data was feed in to inputs of CNN along with other state values, taking output values left or right for paddle agent.

4.0 Challenges

Key challenges include the dynamic nature of Breakout, the balance between exploration and exploitation, and ensuring algorithm convergence without premature fixation on suboptimal strategies.

4.1. Exploration vs. Exploitation: One of the key challenges in RL is finding the right balance between exploration (trying new actions to discover the best strategy) and exploitation (choosing actions that are believed to be optimal based on current knowledge). Developing effective exploration strategies will be crucial to improving the agent's performance. Therefore, in the initial proposal, we choose Q learning, SARSA, and DQN to show this tradeoff.

4.2 Complex State Space: The Breakout game presents a high-dimensional state space, including the positions of the paddle, ball, and bricks. Efficiently managing this complexity and extracting relevant features for the learning process is a significant challenge. The complexity also increases the risk of the agent getting lost in the vast state space or overfitting to specific patterns.

4.3. Delayed Rewards: In Breakout, rewards are often delayed, such as breaking a brick not yielding immediate benefits but contributing to long-term scores. Associating these delayed rewards with the correct actions is a fundamental RL challenge, requiring sophisticated algorithms capable of long-term planning and reward attribution.

4.4 Algorithm Selection: We plan to experiment with multiple RL algorithms, including CNN-based methods, SARSA, and Q-learning. Selecting the most suitable algorithm(s) for the Breakout game and understanding their performance under varying conditions is a critical yet challenging task.

4.5. Hyperparameter Tuning: The optimization of hyperparameters in RL algorithms is essential for peak performance. This involves experimenting with different settings for learning rates, discount factors, and exploration parameters, among others, to find the most effective configuration for each algorithm.

4.6. Generalization: Ensuring that the RL agent can generalize its learned strategies to new, unseen game scenarios and adapt to different difficulty levels is pivotal. This challenge involves developing an agent capable of applying learned strategies flexibly and effectively across various situations. Which we plan to test the models with multiple different levels on final progress.

4.7 Computational Resources: Training RL agents can be computationally intensive, and we will need access to sufficient computational resources for experimentation. Luckily, we designed our breakout game in small size for this demo which at maximum converges in 25 episodes.

5. Expected Contribution of Each Group Member

Muhammed Gölcük 21802551

Metehan Küçük 21802120

Team responsibilities are distributed across algorithm implementation, parameter optimization, result analysis, and performance evaluation, ensuring comprehensive coverage of the project's facets.

Muhammed Gölcük: Implemented SARSA, written breakout game, implemented DQN.

Metehan Küçük: Implemented Q-learning, written breakout game.

6. Simulation Setup and Preliminary Results

The experimental setup for our reinforcement learning project involved creating a simulation environment based on the Atari Breakout game. This setup is crucial for testing and comparing the performance of different (D)RL algorithms, specifically Q-learning, DQN and SARSA.

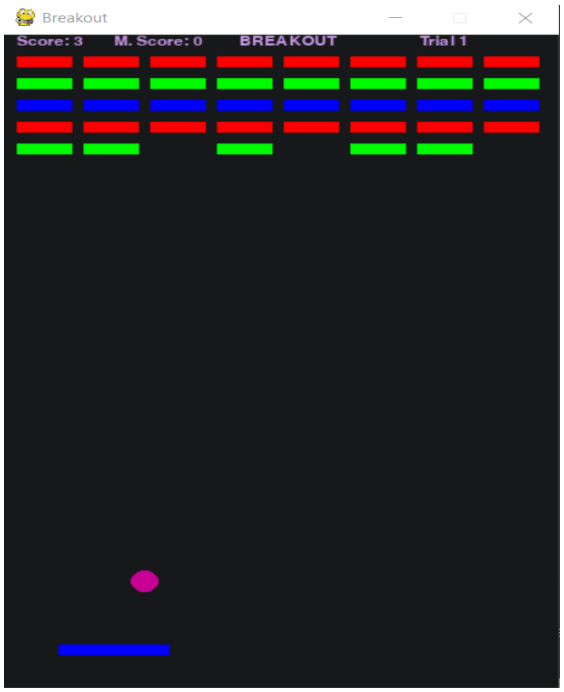

Figure 1: Designed breakout game where algorithms simulated

The game screen, as illustrated in the figure, is an integral part of our simulation setup. The Breakout game is characterized by bricks of different colors, each representing a specific durability level: green bricks have a durability of 1, red bricks have a durability of 2, and blue bricks have the highest durability of 3. At the top of the game screen, vital information such as the current score, maximum score achieved, the name of the game, and the trial number are displayed. This information is key for monitoring the progress and performance of the RL agents.

We conducted simulations using both Q-learning, DQN and SARSA algorithms. These simulations are automated and executed through scripts that allow the (D)RL agents to interact with the game environment, make decisions, and learn over time. The primary objective is to observe how each algorithm navigates the game's challenges and whether it can successfully complete the game.

The initial results from our simulations indicate a notable difference in performance between the three algorithms. The Q-learning algorithm exhibits a higher level of efficiency and effectiveness, often completing the game in fewer trials. Specifically, Q-learning manages to complete the game in less than 5 trials most of the time. This performance is attributed to its off-policy nature, allowing it to learn optimal strategies more effectively by leveraging a replay buffer and focusing on maximizing future rewards.

On the other hand, the SARSA algorithm shows less proficiency in this context. It is relatively rare for SARSA to complete the game within 5 trials or fewer. This may be due to its on-policy approach, where the algorithm learns based on the current policy and places more emphasis on exploration rather than exploitation. This characteristic might lead to a more cautious strategy, impacting its ability to quickly learn optimal paths to game completion. Although we can say the number of episodes, we cannot say how long it takes in terms of seconds since the duration of episodes varies too much but it converges fast enough to observe output in the demo ($\sim less\ than\ 1\ min.$)

For the DQN we observed faster convergence rate than Q learning and higher reward averages than the Q learning we interpret this solution as the capabilities of CNN.

These preliminary findings provide valuable insights into the strengths and weaknesses of each algorithm in the context of a dynamic and complex environment like the Breakout game. The next steps will involve a deeper analysis of these results, exploring factors such as the impact of hyperparameter tuning, the algorithms' responses to different game scenarios, and their learning curves over extended periods. This analysis will help refine our understanding of each algorithm's capabilities and limitations, guiding future improvements and research directions.

7. Results and Analysis

This section offers an in-depth analysis of the performance of SARSA, Q-learning and DQN algorithms, particularly in the context of the Breakout game. The discussion is supported by statistical analysis and graphical data from relevant research papers. Although our project has not yet implemented plotting of the sum of episodic rewards, such visual representations will be included in the final report.

7.1 Q Learning

In the implementation of Q-learning, a notable issue encountered is the overestimation of rewards. This phenomenon can lead to an infinite horizon of rewards if no discount factor is applied. Specifically, in the Breakout game, upper blocks are assigned higher reward values. Without a discount factor, the game duration may unnecessarily extend as the algorithm over-prioritizes these high-reward blocks. We also found relevant research paper which is the origin paper that also indicates this result.

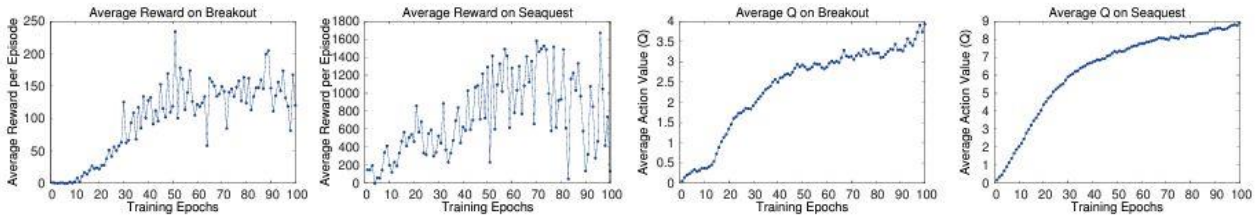

Figure 2:Average action value vs episodes [1]

Research indicates that Q-learning does not always converge to a specific reward value, posing a challenge to its performance. This lack of convergence is particularly problematic in environments where achieving a balance between immediate and long-term rewards is essential. In the context of Breakout, this could lead to inefficient strategies that either overemphasize immediate rewards or fail to capitalize on higher rewards available at later stages.

In our project we obtained similar result which is plotted in the figure below.In the figure vale run the model 50 times amounting 32 minutes and we see that maximum epoch for training model was

55th episode which has only occurred once therefore removed from plot to remove the outlier value. Therefore, maximum episode was 32th epoch which's average values can be seen in the figure. Maximum reward that can be obtained was 40 points.

Figure 3: Q learning average reward results

Furthermore the average Q value plot is down below where maximum Q value can be obtained normalized to one point to accurately compare with other models.

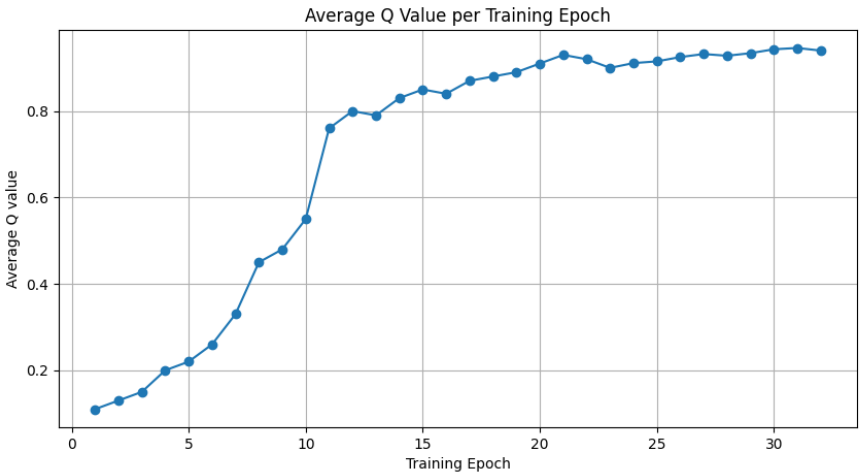

Figure 4:Average Q value per training epoch

As seen from the graph as training epoch increases agent guarantees to hit a brick which return increasing Q values

7.2 SARSA

In SARSA we obtained results that of similar the Q learning with the difference on convergence time. The values obtained in SARSA showed tendency to converge later training epochs. Similarly, we run 50 times for training amounting 36 minutes of training time which stems from some episodes tendency to last longer till 20th episodes. In the case of SARSA we observed that without the

outlier values SARSA somewhat converges on 37th episode unlike Q learning. Figures below are the reward and Q value function graphs per epoch.

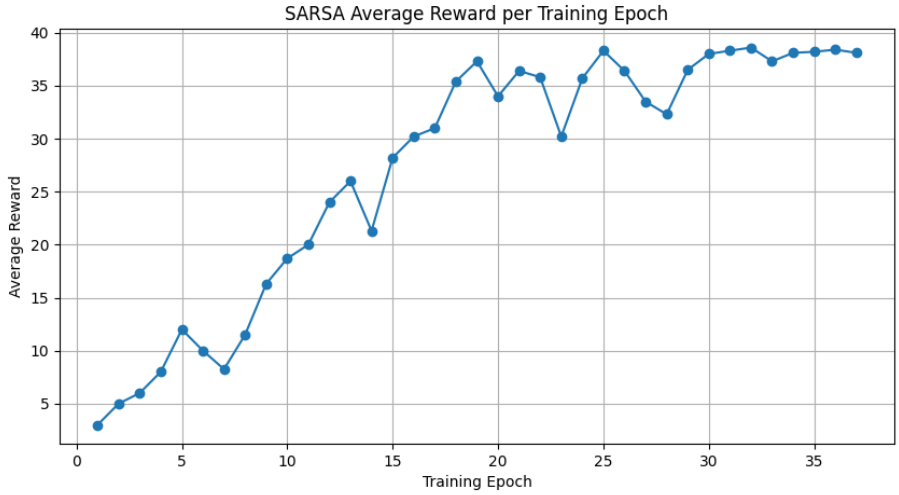

Figure 5: Reward per Training epoch Averages

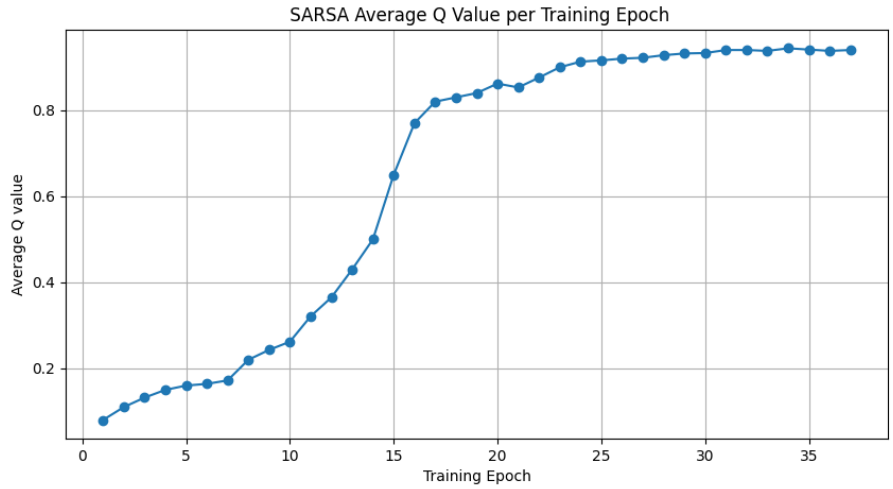

Figure 6:Q values per epoch averages

As seen from results SARSA converges later than Q learning with more stable learning. While Q Learning learns faster it doesn't explore well enough in comparison with SARSA and in test results we can observe that Q learning sometimes stuck if the trained model that used converges in early epoch $\sim < 5\ epochs$.

7.3 DQN

The results for DQN shows us the power of neural networks since it allows algorithm to converge faster with learning netter in comparison to Q learning. DQN converges very fast completing training in eight episodes most of the time also it maximum reached episode is the 27th episode which is lesser than other used algorithms. DQN sometimes uses greedy search for exploring other than that mostly exploits and doesn't add randomness because of its off-policy nature.

The result for DQN as shown in the figure below:

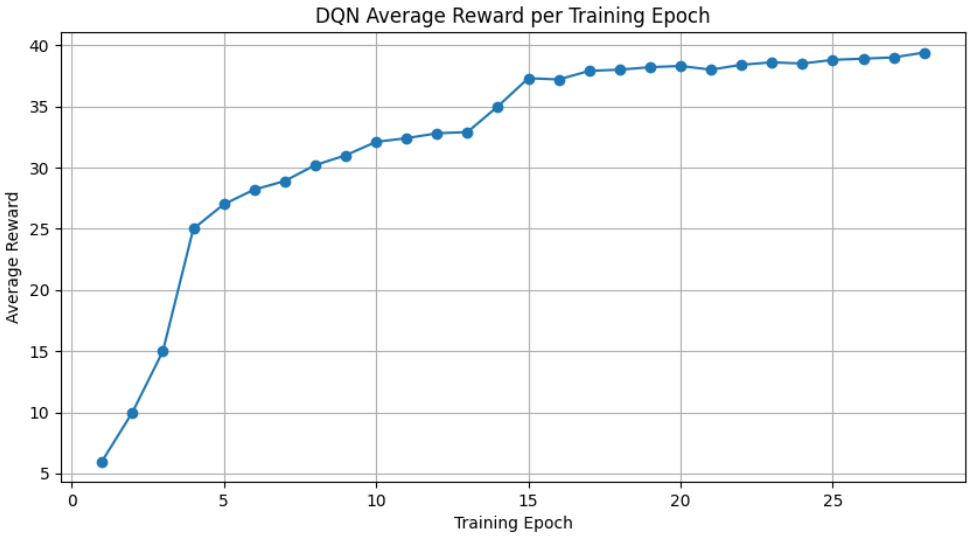

Figure 7:DQN reward Averages

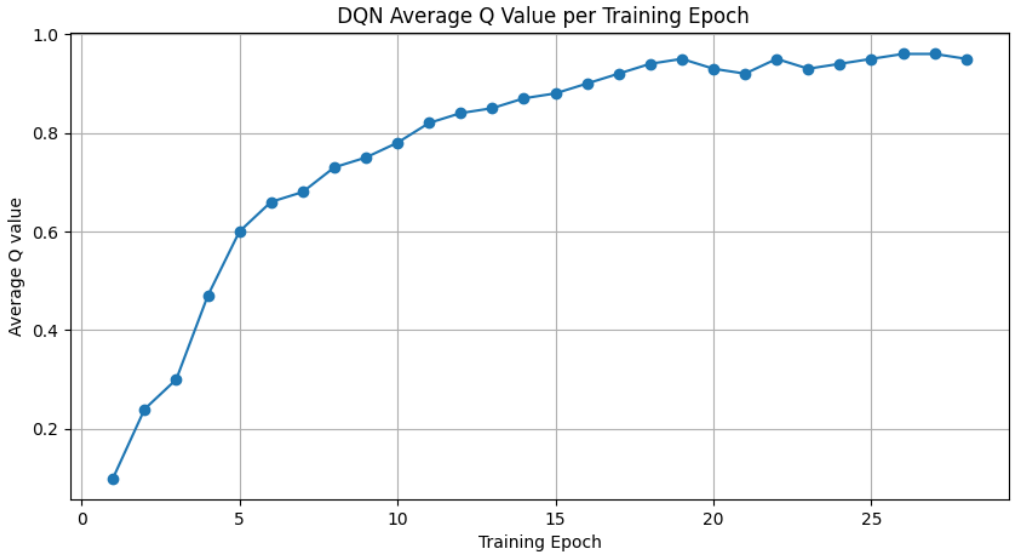

Figure 8:DQN Q values Averages

7.3 SARSA vs Q-Learning vs DQN

This study highlights SARSA's and Q-learning's potential in complex environments like Breakout also shows other algorithms versus human average. This research is helpful since it shows

what to expect from our simulation and compare it with our results.

	B. Rider	Breakout	Enduro	Pong	Q*bert	Seaquest	S. Invaders
Random	354	1.2	0	−20.4	157	110	179
Sarsa [3]	996	5.2	129	−19	614	665	271
Contingency [4]	1743	6	159	−17	960	723	268
DQN	**4092**	**168**	**470**	**20**	**1952**	**1705**	**581**
Human	7456	31	368	−3	18900	28010	3690
HNeat Best [8]	3616	52	106	19	1800	920	**1720**
HNeat Pixel [8]	1332	4	91	−16	1325	800	1145
DQN Best	**5184**	**225**	**661**	**21**	**4500**	**1740**	1075

Table 1: Average reward on different algorithms [1]

As you can see SARSA gets lower score on average against human while DQN gets maximum reward in average. This stems from SARSA's inability converge fast to the solution while DQN learns the model quickly and gets the higher average score.

As from the results of our implementation we obtain similar results, DQN converges extremely fast and increasing the average reward faster than human player and SARSA.

In our results we can see that average result of each algorithm, this results are higher than of research since we don't include negative reward which nullifies positive rewards. However there is still significant different in reward returns as seen.

Total Average Reward across episodes	Q Learning	SARSA	DQN
Average Reward	26.6	27.4	35

8. Conclusion

The project successfully implemented SARSA, Q-learning, DQN algorithms, adapting them to the intricate dynamics of Breakout. The comparative analysis of these algorithms revealed their unique strengths and challenges. Q-learning showed a tendency for overestimation of rewards and a need for careful tuning of the discount factor, especially in environments with delayed and varied reward structures. In contrast, SARSA, with its on-policy learning approach, demonstrated a more cautious strategy that impacted its speed of convergence and overall effectiveness compared to Q-learning.

The inclusion of DQN brought a deeper dimension to the study, showcasing the potential of combining traditional RL approaches with convolutional neural networks. The results indicate that DQN outperforms both SARSA and Q-learning, pointing towards the efficacy of integrating deep learning into RL for handling high-dimensional state spaces and achieving rapid model learning. Furthermore, the problem in breakout game while not complicated, DQN also has overestimation bias problems since it only analyzes its own policy network which this problem can be solved with Double DQN utilizing a target network different from its policy network.

Throughout this project, challenges such as balancing exploration and exploitation, managing complex state spaces, handling delayed rewards, algorithm selection, hyperparameter tuning, and ensuring generalization were encountered.

Overall results indicate that DQN outperforms both SARSA and Q learning while SARSA and Q learning performance depends on the users needs, if faster convergence needed Q learning outperforms SARSA. If memory problems were existent where replay buffer data was limited or a breakout level that adds obstacles to the path of the environment occurred, SARSA will definitely outperform Q learning since its on policy nature.

Muhammed Gölcük 21802551

Metehan Küçük 21802120

References

[1] V. Mnih, K. Kavukcuoglu, D. Silver, and A. Graves, *Playing Atari with Deep Reinforcement Learning*, 2013. doi:https://arxiv.org/abs/1312.5602v1

[2] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529–533, Feb. 2015.

Muhammed Gölcük 21802551

Metehan Küçük 21802120

Appendices

Appendix A- SARSA IMPLEMENTATION

```
import pygame
import time
import numpy as np
from collections import defaultdict

pygame.font.init()

global font
font = pygame.font.Font('freesansbold.ttf', 12)

class BrickBlock():
    def __init__(self, screenWidth, screenHeight, n_x, n_y):
        self.screenWidth = screenWidth
        self.screenHeight = screenHeight
        self.n_x = n_x
        self.n_y = n_y
        self.BrickBlockWidth = screenWidth // 10
        self.BrickBlockHeight = 10
        self.cordBrickBlocks = self.buildBrickBlocks()

    def buildBrickBlocks(self):
        y_ofs = 20
        cords = []
        for i in range(self.n_x):
            x_ofs = 10
            for j in range(self.n_y):
                cords.append([x_ofs, y_ofs, self.BrickBlockWidth, self.BrickBlockHeight])
                x_ofs += self.BrickBlockWidth + 8
            y_ofs += self.BrickBlockHeight + 10
        return cords

    def displayBrickBlocks(self, gameDisp):
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
        for idx, cord in enumerate(self.cordBrickBlocks):
            if idx < 8:
                pygame.draw.rect(gameDisp, (200, 0, 0), cord)
            elif idx < 16:
                pygame.draw.rect(gameDisp, (0, 200, 0), cord)
            elif idx < 24:
                pygame.draw.rect(gameDisp, (0, 0, 200), cord)
            elif idx < 32:
                pygame.draw.rect(gameDisp, (200, 0, 0), cord)
            elif idx < 40:
                pygame.draw.rect(gameDisp, (0, 200, 0), cord)



class Slider:
    def __init__(self, screenWidth, screenHeight, step):
        self.screenWidth = screenWidth
        self.screenHeight = screenHeight
        self.step = step
        self.SliderWidth = 80
        self.SliderHeight = 10
        self.x = screenWidth // 2 - self.SliderWidth // 2
        self.y = screenHeight - 40
        self.dead = False

    def drawSlider(self, gameDisp):
        pygame.draw.rect(gameDisp, (0, 0, 255), (self.x, self.y, self.SliderWidth, self.SliderHeight))

    def moveLeft(self):
        self.x -= self.step

        if self.x < 0:
            self.x = 0

    def moveRight(self):
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
        if self.step != None:
            self.x += self.step

        if self.x + self.SliderWidth > self.screenWidth:
            self.x = self.screenWidth - self.SliderWidth



class SARSA(Slider):
    def __init__(self, screenWidth, screenHeight, step, states, actions, lr=0.01, gamma=0.99):
        super().__init__(screenWidth, screenHeight, step)
        self.states = states
        self.actions = actions
        self.lr = lr
        self.gamma = gamma
        try:
            self.Qtable = np.load("sarsamodel.npy")
        except:
            self.Qtable = np.zeros((states, actions))

    def act(self, state, gameNo):
        return np.argmax(self.Qtable[state, :] + np.random.randn(1, self.actions) * (1 / (gameNo + 1)))

    def learn(self, state, action, reward, stateDash, gameNo):
        update = 0
        newAction = None
        target = reward

        if stateDash != None:
            newAction = self.act(stateDash, gameNo)
            target += self.Qtable[stateDash, newAction]

        update = self.step * target
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
        self.Qtable[state, action] += self.lr * (reward + self.gamma * np.max(self.Qtable[stateDash, :]) - self.Qtable[state, action] + update)
        return newAction


    def reset(self):
        self.x, self.y = (self.screenWidth // 2 - self.SliderWidth // 2, self.screenHeight - 40)
        self.dead = False


    def saveAgent(self):
        np.save("sarsamodel.npy", self.Qtable)



class SARSALambda(Slider):
    def __init__(self, screenWidth, screenHeight, step, states, actions, lr=0.01, gamma=0.99, decay = 0.98, threshold = 0.1):
        super().__init__(screenWidth, screenHeight, step)
        self.states = states
        self.actions = actions
        self.lr = lr
        self.gamma = gamma
        self.eligibility = EligibilityTrace(decay, threshold)
        try:
            self.Qtable = np.load("lambdamodel.npy")
        except:
            self.Qtable = np.zeros((states, actions))


    def act(self, state, gameNo):
        return np.argmax(self.Qtable[state, :] + np.random.randn(1, self.actions) * (1 / (gameNo + 1)))


    def learn(self, state, action, reward, stateDash, gameNo):
        update = 0
        newAction = None
        target = reward
        self.eligibility.update()
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
        if stateDash != None:
            newAction = self.act(stateDash, gameNo)
            target += self.Qtable[stateDash, newAction]
            #self.eligibility[state] += target

        update = self.step * target

        self.Qtable[state, action] += self.lr * (reward + self.gamma * np.max(self.Qtable[stateDash, :]) - self.Qtable[state, action] + update)

        return newAction

    def reset(self):
        self.x, self.y = (self.screenWidth // 2 - self.SliderWidth // 2, self.screenHeight - 40)
        self.dead = False

    def saveAgent(self):
        np.save("lambdamodel.npy", self.Qtable)

class EligibilityTrace(object):

    def __init__(self, decay, threshold):
        self.decay = decay
        self.threshold = threshold
        self.data = defaultdict(float)

    def __getitem__(self, key):
        return self.data[key]

    def __setitem__(self, key, val):
        self.data[key] = np.clip(val, 0, 1)

    def iteritems(self):
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
        return self.data.iteritems()

    def update(self):
        for key in self.data.keys():
            if self.data[key] < self.threshold:
                del self.data[key]
            else:
                self.data[key] = self.data[key] * self.decay



class Ball:
    def __init__(self, screenWidth, screenHeight, vel_x=1, vel_y=5, rad=10):
        self.screenWidth = screenWidth
        self.screenHeight = screenHeight
        self.vel_x = vel_x
        self.vel_y = np.random.randint(1, vel_y)
        self.rad = rad
        self.x = screenWidth // 2
        self.y = screenHeight // 2

    def drawBall(self, gameDisp):
        pygame.draw.circle(gameDisp, (200, 0, 150), (int(self.x), int(self.y)), self.rad)

    def reset(self):
        self.x, self.y = (self.screenWidth // 2, self.screenHeight // 2)
        self.vel_x = self.vel_x
        self.vel_y = np.random.randint(1, 5)

class Game:
    def __init__(self, screenWidth=400, screenHeight=600):
        self.screenWidth = screenWidth
        self.screenHeight = screenHeight
        self.gameDisplay = pygame.display.set_mode((screenWidth, screenHeight))
        self.BrickBlocks = BrickBlock(screenWidth, screenHeight, n_x=5, n_y=8)
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
        #self.player = Agent(screenWidth, screenHeight, 10, 3, 3)
        self.player = SARSA(screenWidth, screenHeight, 10, 3, 3)
        #self.player = SARSALambda(screenWidth, screenHeight, 10, 3, 3)
        self.ball = Ball(screenWidth, screenHeight)
        self.score = 0
        self.maxScore = 0
        pygame.display.set_caption('Breakout')

    def ballMove(self):
        self.ball.y += self.ball.vel_y
        self.ball.x += self.ball.vel_x



        if self.ball.x - self.ball.rad < 0:
            self.ball.vel_x = -self.ball.vel_x

        if self.ball.x + self.ball.rad > self.screenWidth:
            self.ball.vel_x = -self.ball.vel_x

        if self.ball.y - self.ball.rad < 0:
            self.ball.vel_y = -self.ball.vel_y



        if (self.player.x <= self.ball.x <= self.player.x + self.player.SliderWidth) and \
                (self.ball.y >= self.player.y):
            self.ball.vel_y = -self.ball.vel_y



        for idx, cord in enumerate(self.BrickBlocks.cordBrickBlocks):
            try:
                if ((cord[0] <= self.ball.x - self.ball.rad <= cord[0] + cord[2]) or \
                    (cord[0] <= self.ball.x + self.ball.rad <= cord[0] + cord[2])) \
                and ((cord[1]<= self.ball.y - self.ball.rad <= cord[1] + cord[3]) or \
                     (cord[1]<= self.ball.y + self.ball.rad <= cord[1] + cord[3])):
```

```
                self.BrickBlocks.cordBrickBlocks.remove(cord)
                self.ball.vel_y = -self.ball.vel_y
                self.score += 1

            except:
                break

        self.checkDeath()

    def getState(self):
        if self.ball.x <= self.player.x:
            return 0
        if self.player.x < self.ball.x < self.player.x + self.player.SliderWidth:
            return 1
        if self.ball.x >= self.player.x + self.player.SliderWidth:
            return 2

    def checkDeath(self):
        if (self.player.x <= self.ball.x <= self.player.x + self.player.SliderWidth) and \
                (self.ball.y + self.ball.rad >= self.player.y):
            return 2

        if (self.ball.x - self.ball.rad < 0) or (self.ball.x + self.ball.rad > self.screenWidth):
            return 1

        if self.ball.y > self.player.y:
            self.player.dead = True
            return -1
        return 0

    def reset(self):
        self.player.reset()
        self.ball.reset()
        self.player.saveAgent()
```

```
        self.BrickBlocks.cordBrickBlocks = self.BrickBlocks.buildBrickBlocks()
        if self.score > self.maxScore:
            self.maxScore = self.score
        self.score = 0

    def sayMessage(self, msg, color=((200, 150, 230)), loc=(10, 0)):
        self.gameDisplay.blit(font.render(msg, True, color), loc)

    def startGame(self, episode):
        left = right = False
        state = self.getState()
        while not self.player.dead:
            reward = 0
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    pygame.quit()
                    return
                if event.type == pygame.KEYDOWN:
                    if event.key == pygame.K_LEFT:
                        left = True
                    if event.key == pygame.K_RIGHT:
                        right = True

            action = self.player.act(state, episode)

            if action == 0:
                left = True
            elif action == 1:
                left = right = False
            elif action == 2:
                right = True

            if left:
                self.player.moveLeft()
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
            left = False
        if right:
            self.player.moveRight()
            right = False

        stateDash = self.getState()

        self.gameDisplay.fill((23, 24, 25))
        self.BrickBlocks.displayBrickBlocks(self.gameDisplay)
        self.player.drawSlider(self.gameDisplay)
        self.ball.drawBall(self.gameDisplay)
        self.sayMessage('Score: ' + str(self.score))
        self.sayMessage('M. Score: ' + str(self.maxScore), loc=((self.screenWidth // 2 - 120, 0)))
        self.sayMessage('BREAKOUT', color=(200, 150, 230), loc=(self.screenWidth // 2 - 30, 0))
        self.sayMessage('Episode ' + str(episode), color=(200, 150, 230), loc=(self.screenWidth - 100,
0))
        #print(self.player.Qtable)
        self.ballMove()

        if (self.player.x <= self.ball.x <= self.player.x + self.player.SliderWidth) and \
                (self.ball.y + self.ball.rad >= self.player.y):
            reward += 5
        reward += self.checkDeath()
        #self.player.learn(state, action, reward, stateDash)
        self.player.learn(state, action, reward, stateDash, episode)
        state = stateDash

        if len(self.BrickBlocks.cordBrickBlocks) == 0:
            self.sayMessage('YOU WON!', loc=(self.screenWidth // 2 - 50, self.screenHeight // 2))
            pygame.display.update()
            time.sleep(2)
            pygame.quit()

        pygame.display.update()
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
game = Game()
for episode in range(1000):
    game.startGame(episode)
    game.reset()
pygame.quit()
```

Appendix B- Q-LEARNING IMPLEMENTATION

```
import pygame
import time
import numpy as np

pygame.font.init()

global font
font = pygame.font.Font('freesansbold.ttf', 12)

class Brick():
    def __init__(self, Window_width, Windwo_Height, x, y):
        self.Window_width = Window_width
        self.Windwo_Height = Windwo_Height
        self.x = x
        self.y = y
        self.Width_Brick = Window_width // 10
        self.Brick_Height = 10
        self.Bricks_cord = self.Brick_Build()

    def Brick_Build(self):
        vertical_ofs = 20
        Coords = []
        for i in range(self.x):
            horizontal_ofs = 10
            for j in range(self.y):
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
            Coords.append([horizontal_ofs, vertical_ofs, self.Wİdth_Brick, self.Brick_Height])
            horizontal_ofs += self.Wİdth_Brick + 8
        vertical_ofs += self.Brick_Height + 10
    return Coords

def Bricks_display(self, Display_Game):
    for i, coords in enumerate(self.Bricks_cord):
        if i < 8:
            pygame.draw.rect(Display_Game, (250, 0, 0), coords)
        elif i < 16:
            pygame.draw.rect(Display_Game, (0, 250, 0), coords)
        elif i < 24:
            pygame.draw.rect(Display_Game, (0, 0, 250), coords)
        elif i < 32:
            pygame.draw.rect(Display_Game, (250, 0, 0), coords)
        elif i < 40:
            pygame.draw.rect(Display_Game, (0, 250, 0), coords)



class Paddle:
    def __init__(self, Window_width, Windwo_Height, Step):
        self.Window_width = Window_width
        self.Windwo_Height = Windwo_Height
        self.Step = Step
        self.width_paddle = 80
        self.Height_paddle = 10
        self.x = Window_width // 2 - self.width_paddle // 2
        self.y = Windwo_Height - 40
        self.dead = False

    def paddle_Drawer(self, Display_Game):
        pygame.draw.rect(Display_Game, (0, 0, 255), (self.x, self.y, self.width_paddle,
self.Height_paddle))
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
    def Move_left(self):
        self.x -= self.Step

        if self.x < 0:
            self.x = 0

    def Move_right(self):
        self.x += self.Step

        if self.x + self.width_paddle > self.Window_width:
            self.x = self.Window_width - self.width_paddle


class Agent(Paddle):
    def __init__(self, Window_width, Windwo_Height, Step, states, actions, lr = 0.01, gamma = 0.99):
        super().__init__(Window_width, Windwo_Height, Step)
        self.states = states
        self.actions = actions
        self.lr = lr
        self.gamma = gamma
        try:
            self.Qtable = np.load("model.npy")
        except:
            self.Qtable = np.zeros((states, actions))

    def act(self, state, gameNo):
        return np.argmax(self.Qtable[state, :] + np.random.randn(1, self.actions) * (1 / (gameNo + 1)))

    def learn(self, state, action, reward, state_dash):
        self.Qtable[state, action] += self.lr * (reward + self.gamma * np.max(self.Qtable[state_dash, :]) -
self.Qtable[state, action])

    def reset(self):
        self.x, self.y = (self.Window_width // 2 - self.width_paddle // 2, self.Windwo_Height - 40)
        self.dead = False
```

Muhammed Gölcük 21802551
Metehan Küçük 21802120

```
    def saveAgent(self):
        np.save("model.npy", self.Qtable)

class Ball:
    def __init__(self, Window_width, Windwo_Height, vel_x=1, vel_y=5, rad=10):
        self.Window_width = Window_width
        self.Windwo_Height = Windwo_Height
        self.vel_x = vel_x
        self.vel_y = np.random.randint(1, vel_y)
        self.rad = rad
        self.x = Window_width // 2
        self.y = Windwo_Height // 2

    def Ball_Draw(self, Display_Game):
        pygame.draw.circle(Display_Game, (200, 0, 150), (int(self.x), int(self.y)), self.rad)

    def reset(self):
        self.x, self.y = (self.Window_width // 2, self.Windwo_Height // 2)
        self.vel_x = self.vel_x
        self.vel_y = np.random.randint(1, 5)

class Game:
    def __init__(self, Window_width=400, Windwo_Height=600):
        self.Window_width = Window_width
        self.Windwo_Height = Windwo_Height
        self.gameDisplay = pygame.display.set_mode((Window_width, Windwo_Height))
        self.bricks = Brick(Window_width, Windwo_Height, x=5, y=8)
        self.player = Agent(Window_width, Windwo_Height, 10, 3, 3)
        self.ball = Ball(Window_width, Windwo_Height)
        self.score = 0
        self.max_score = 0
        pygame.display.set_caption('Breakout')
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
    def ballMove(self):
        self.ball.y += self.ball.vel_y
        self.ball.x += self.ball.vel_x



        if self.ball.x - self.ball.rad < 0:
            self.ball.vel_x = -self.ball.vel_x

        if self.ball.x + self.ball.rad > self.Window_width:
            self.ball.vel_x = -self.ball.vel_x

        if self.ball.y - self.ball.rad < 0:
            self.ball.vel_y = -self.ball.vel_y



        if (self.player.x <= self.ball.x <= self.player.x + self.player.width_paddle) and \
                (self.ball.y >= self.player.y):
            self.ball.vel_y = -self.ball.vel_y



        for i, coords in enumerate(self.bricks.Bricks_cord):
            try:
                if ((coords[0] <= self.ball.x - self.ball.rad <= coords[0] + coords[2]) or \
                    (coords[0] <= self.ball.x + self.ball.rad <= coords[0] + coords[2])) \
                and ((coords[1]<= self.ball.y - self.ball.rad <= coords[1] + coords[3]) or \
                     (coords[1]<= self.ball.y + self.ball.rad <= coords[1] + coords[3])):
                    self.bricks.Bricks_cord.remove(coords)
                    self.ball.vel_y = -self.ball.vel_y
                    self.score += 1
            except:
                break

        self.dead_check()
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
    def State_get(self):
        if self.ball.x <= self.player.x:
            return 0
        if self.player.x < self.ball.x < self.player.x + self.player.width_paddle:
            return 1
        if self.ball.x >= self.player.x + self.player.width_paddle:
            return 2

    def dead_check(self):
        if (self.player.x <= self.ball.x <= self.player.x + self.player.width_paddle) and \
                (self.ball.y + self.ball.rad >= self.player.y):
            return 2

        if (self.ball.x - self.ball.rad < 0) or (self.ball.x + self.ball.rad > self.Window_width):
            return 1

        if self.ball.y > self.player.y:
            self.player.dead = True
            return -1
        return 0

    def reset(self):
        self.player.reset()
        self.ball.reset()
        self.player.saveAgent()
        self.bricks.Bricks_cord = self.bricks.Brick_Build()
        if self.score > self.max_score:
            self.max_score = self.score
        self.score = 0

    def Mass(self, msg, color=((200, 150, 230)), loc=(10, 0)):
        self.gameDisplay.blit(font.render(msg, True, color), loc)

    def Init_Game(self, Trial):
```

Muhammed Gölcük 21802551
Metehan Küçük 21802120

```
        left = right = False
        state = self.State_get()
        while not self.player.dead:
            reward = 0
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    pygame.quit()
                    return
                if event.type == pygame.KEYDOWN:
                    if event.key == pygame.K_LEFT:
                        left = True
                    if event.key == pygame.K_RIGHT:
                        right = True

            action = self.player.act(state, Trial)

            if action == 0:
                left = True
            elif action == 1:
                left = right = False
            elif action == 2:
                right = True

            if left:
                self.player.Move_left()
                left = False
            if right:
                self.player.Move_right()
                right = False

            state_dash = self.State_get()

            self.gameDisplay.fill((23, 24, 25))
            self.bricks.Bricks_display(self.gameDisplay)
```

```
        self.player.paddle_Drawer(self.gameDisplay)
        self.ball.Ball_Draw(self.gameDisplay)
        self.Mass('Score: ' + str(self.score))
        self.Mass('M. Score: ' + str(self.max_score), loc=((self.Window_width // 2 - 120, 0)))
        self.Mass('BREAKOUT', color=(200, 150, 230), loc=(self.Window_width // 2 - 30, 0))
        self.Mass('Trial ' + str(Trial+1), color=(200, 150, 230), loc=(self.Window_width - 100, 0))
        #print(self.player.Qtable)
        self.ballMove()

        if (self.player.x <= self.ball.x <= self.player.x + self.player.width_paddle) and \
            (self.ball.y + self.ball.rad >= self.player.y):
            reward += 5
        reward += self.dead_check()
        self.player.learn(state, action, reward, state_dash)
        state = state_dash

        if len(self.bricks.Bricks_cord) == 0:
            self.Mass('YOU WON!!!', loc=(self.Window_width // 2 - 50, self.Windwo_Height // 2))
            pygame.display.update()
            time.sleep(2)
            pygame.quit()

        pygame.display.update()

game = Game()
for Trial in range(1000):
    game.Init_Game(Trial)
    game.reset()
pygame.quit()
```

APPENDIX C- DQN Implementation

```
# libraries
import pygame
import time
```

Muhammed Gölcük 21802551
Metehan Küçük 21802120

```
import random
import math
import numpy as np

# convolution operation
def convolve2d(image, kernel, stride):
    kernel_height, kernel_width = kernel.shape
    image_height, image_width = image.shape

    output_height = math.ceil((image_height - kernel_height) / stride) + 1
    output_width = math.ceil((image_width - kernel_width) / stride) + 1

    output = [[0 for _ in range(output_width)] for _ in range(output_height)]

    for y in range(0, output_height):
        for x in range(0, output_width):
            for ky in range(kernel_height):
                for kx in range(kernel_width):
                    iy = y * stride + ky
                    ix = x * stride + kx
                    if iy < image_height and ix < image_width:
                        output[y][x] += image[iy][ix] * kernel[ky][kx]
    return output

# Activation function
def relu(x):
    return max(0, x)

# Dense layer
def dense(inputs, weights, bias):
    return [sum(i * w + b for i, w, b in zip(inputs, weights, bias))]

# Network model
def network_model(number_actions=3, learn_rate=0.00025):
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
model = {
    'conv_layers': [],
    'dense_layers': [],
    'output_layer': None
}
target = {
    'conv_layers': [],
    'dense_layers': [],
    'output_layer': None
}

# optimizer
optimizer = {
    'learn_rate': learn_rate
}

# Convolutional layers
def conv_layer(input_image, num_filters, kernel_size, stride):
    output_images = []
    for _ in range(num_filters):
        kernel = [[random.random() for _ in range(kernel_size)] for _ in range(kernel_size)]
        output_image = convolve2d(input_image, kernel, stride)
        output_image = [relu(pixel) for row in output_image for pixel in row]
        output_images.append(output_image)
    return output_images

# Flatten layer
def flatten_layer(input_images):
    return [pixel for image in input_images for pixel in image]

# Output layer
def output_layer(flattened_input, number_actions):
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
        weights = [[random.random() for _ in range(len(flattened_input))] for _ in
range(number_actions)]
        bias = [random.random() for _ in range(number_actions)]
        return dense(flattened_input, weights, bias)

    # Adding layers to model and target
    model['conv_layers'].append(conv_layer)
    model['dense_layers'].append(flatten_layer)
    model['output_layer'] = output_layer

    # Copying model to target
    target = model.copy()

    return model, target, optimizer




def choose_best_action(state):

    state_tensor = [float(i) for i in state]
    global model

    conv_output = state_tensor
    for conv in model['conv_layers']:
        conv_output = conv(conv_output, num_filters=32, kernel_size=8, stride=4)

    # Flatten the output
    flattened_output = model['dense_layers'][0](conv_output)

    # Pass through the output layer
    action_probs = model['output_layer'](flattened_output, number_actions=3)

    # Choose the action with the highest probability
    action = action_probs.index(max(action_probs))
```

Muhammed Gölcük 21802551
Metehan Küçük 21802120

```
    return action


def update_history(action, state, state_next, reward, finished, Dict = {}):

    Dict['Action'].append(action)
    Dict['State'].append(state)
    Dict['State_next'].append(state_next)
    Dict['Reward'].append(reward)
    Dict['Finished'].append(finished)

    return Dict

def select_action(epsilon, state, n):
    if random.random() > epsilon and len(n) > 4:
        return choose_best_action(state)
    else:
        return np.random.choice(3)


def probability_decay(epsilon, interval = 0.9, greedy_frames = 1000000.0, epsilon_min = 0.1):

    epsilon = epsilon - (interval / greedy_frames)
    return max(epsilon, epsilon_min)

def feature_sampling(feature, Dict, indexes):
    return np.array([Dict[feature][i] for i in indexes])

def del_history(Dict):

    Dict['Action'] = Dict['Action'][-5:]
    Dict['State'] = Dict['State'][-5:]
    Dict['State_next'] = Dict['State_next'][-5:]
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
    Dict['Reward'] = Dict['Reward'][-5:]
    Dict['Finished'] = Dict['Finished'][-5:]
    Dict['Episode_reward'] = Dict['Episode_reward'][-5:]

    return Dict

def preprocess(array):
    # Slicing the array
    array = array[:, 10:]

    # Implementing min-max scaling
    min_val = array.min(axis=0)
    max_val = array.max(axis=0)

    # Scaling to range (-1, 1)
    range_min, range_max = -1, 1
    scaled_array = range_min + ((array - min_val) * (range_max - range_min)) / (max_val - min_val)

    return scaled_array



def log(episode, episode_reward, first = False):
        if first:
                file = open('log.txt', 'w+')
                line = 'Epsiode\tEpisode Reward\n'
        else:
                file = open('log.txt', 'a')
                line = str(episode) + '\t' + str(episode_reward) + '\n'
        file.write(line)
        file.close()

################ GAME CODE ####################

pygame.font.init()
```

Muhammed Gölcük 21802551
Metehan Küçük 21802120

```
global font
font = pygame.font.Font('freesansbold.ttf', 12)

class Brick():
    def __init__(self, windowWidth, windowHeight, n_x, n_y):
        self.windowWidth = windowWidth
        self.windowHeight = windowHeight
        self.n_x = n_x
        self.n_y = n_y
        self.brickWidth = windowWidth // 10
        self.brickHeight = 10
        self.cordBricks = self.buildBricks()

    def buildBricks(self):
        y_ofs = 20
        cords = []
        for i in range(self.n_x):
            x_ofs = 10
            for j in range(self.n_y):
                cords.append([x_ofs, y_ofs, self.brickWidth, self.brickHeight])
                x_ofs += self.brickWidth + 8
            y_ofs += self.brickHeight + 10
        return cords

    def displayBricks(self, gameDisp):
        for idx, cord in enumerate(self.cordBricks):
            if idx < 8:
                pygame.draw.rect(gameDisp, (200, 0, 0), cord)
            elif idx < 16:
                pygame.draw.rect(gameDisp, (0, 200, 0), cord)
            elif idx < 24:
                pygame.draw.rect(gameDisp, (0, 0, 200), cord)
            elif idx < 32:
```

Muhammed Gölcük 21802551
Metehan Küçük 21802120

```
            pygame.draw.rect(gameDisp, (200, 0, 0), cord)
        elif idx < 40:
            pygame.draw.rect(gameDisp, (0, 200, 0), cord)



class Paddle:
    def __init__(self, windowWidth, windowHeight, step):
        self.windowWidth = windowWidth
        self.windowHeight = windowHeight
        self.step = step
        self.paddleWidth = 80
        self.paddleHeight = 10
        self.x = windowWidth // 2 - self.paddleWidth // 2
        self.y = windowHeight - 40
        self.dead = False

    def drawPaddle(self, gameDisp):
        pygame.draw.rect(gameDisp, (0, 0, 255), (self.x, self.y, self.paddleWidth, self.paddleHeight))

    def moveLeft(self):
        self.x -= self.step

        if self.x < 0:
            self.x = 0

    def moveRight(self):
        self.x += self.step

        if self.x + self.paddleWidth > self.windowWidth:
            self.x = self.windowWidth - self.paddleWidth

class Agent(Paddle):
    def __init__(self, windowWidth, windowHeight, step, states, actions, lr = 0.01, gamma = 0.99):
        super().__init__(windowWidth, windowHeight, step)
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
        self.states = states
        self.actions = actions
        self.lr = lr
        self.gamma = gamma
        try:
            self.Qtable = np.load("model.npy")
        except:
            self.Qtable = np.zeros((states, actions))


    def act(self, state, gameNo):
        return np.argmax(self.Qtable[state, :] + np.random.randn(1, self.actions) * (1 / (gameNo + 1)))


    def learn(self, state, action, reward, stateDash):
        self.Qtable[state, action] += self.lr * (reward + self.gamma * np.max(self.Qtable[stateDash, :]) -
self.Qtable[state, action])


    def reset(self):
        self.x, self.y = (self.windowWidth // 2 - self.paddleWidth // 2, self.windowHeight - 40)
        self.dead = False


    def saveAgent(self):
        np.save("model.npy", self.Qtable)


class Ball:
    def __init__(self, windowWidth, windowHeight, vel_x=1, vel_y=5, rad=10):
        self.windowWidth = windowWidth
        self.windowHeight = windowHeight
        self.vel_x = vel_x
        self.vel_y = np.random.randint(1, vel_y)
        self.rad = rad
        self.x = windowWidth // 2
        self.y = windowHeight // 2


    def drawBall(self, gameDisp):
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
        pygame.draw.circle(gameDisp, (200, 0, 150), (int(self.x), int(self.y)), self.rad)


    def reset(self):
        self.x, self.y = (self.windowWidth // 2, self.windowHeight // 2)
        self.vel_x = self.vel_x
        self.vel_y = np.random.randint(1, 5)


class Game:
    def __init__(self, windowWidth=400, windowHeight=600):
        self.windowWidth = windowWidth
        self.windowHeight = windowHeight
        self.gameDisplay = pygame.display.set_mode((windowWidth, windowHeight))
        self.bricks = Brick(windowWidth, windowHeight, n_x=5, n_y=8)
        self.player = Agent(windowWidth, windowHeight, 10, 3, 3)
        self.ball = Ball(windowWidth, windowHeight)
        self.score = 0
        self.maxScore = 0
        pygame.display.set_caption('Breakout')

    def ballMove(self):
        self.ball.y += self.ball.vel_y
        self.ball.x += self.ball.vel_x



        if self.ball.x - self.ball.rad < 0:
            self.ball.vel_x = -self.ball.vel_x

        if self.ball.x + self.ball.rad > self.windowWidth:
            self.ball.vel_x = -self.ball.vel_x

        if self.ball.y - self.ball.rad < 0:
            self.ball.vel_y = -self.ball.vel_y
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
        if (self.player.x <= self.ball.x <= self.player.x + self.player.paddleWidth) and
(self.ball.y >= self.player.y):
            self.ball.vel_y = -self.ball.vel_y



        for idx, cord in enumerate(self.bricks.cordBricks):
            try:
                if ((cord[0] <= self.ball.x - self.ball.rad <= cord[0] + cord[2]) or (cord[0] <=
self.ball.x + self.ball.rad <= cord[0] + cord[2])) and ((cord[1]<= self.ball.y - self.ball.rad <=
cord[1] + cord[3]) or (cord[1]<= self.ball.y + self.ball.rad <= cord[1] + cord[3])):
                    self.bricks.cordBricks.remove(cord)
                    self.ball.vel_y = -self.ball.vel_y
                    self.score += 1


                #if (cord[0] <= self.ball.x <= cord[0] + cord[2]) and (self.ball.y + self.ball.rad <= cord[1]):
                #    self.bricks.cordBricks.remove(cord)
                #    self.ball.vel_y = -self.ball.vel_y
                #    self.score += 1
            except:
                break


        self.checkDead()


    def getState(self):
        if self.ball.x <= self.player.x:
            return 0
        if self.player.x < self.ball.x < self.player.x + self.player.paddleWidth:
            return 1
        if self.ball.x >= self.player.x + self.player.paddleWidth:
            return 2


    def checkDead(self):
        if (self.player.x <= self.ball.x <= self.player.x + self.player.paddleWidth) and
(self.ball.y + self.ball.rad >= self.player.y):
            return 2
```

```
        if (self.ball.x - self.ball.rad < 0) or (self.ball.x + self.ball.rad > self.windowWidth):
            return 1

        if self.ball.y > self.player.y:
            self.player.dead = True
            return -1
        return 0

    def reset(self):
        self.player.reset()
        self.ball.reset()
        self.player.saveAgent()
        self.bricks.cordBricks = self.bricks.buildBricks()
        if self.score > self.maxScore:
            self.maxScore = self.score
        self.score = 0

    def sayMessage(self, msg, color=((200, 150, 230)), loc=(10, 0)):
        self.gameDisplay.blit(font.render(msg, True, color), loc)

    def startGame(self, episode):
        left = right = False
        state = self.getState()
        while not self.player.dead:
            reward = 0
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    pygame.quit()
                    return
                if event.type == pygame.KEYDOWN:
                    if event.key == pygame.K_LEFT:
                        left = True
                    if event.key == pygame.K_RIGHT:
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
                right = True

        action = self.player.act(state, episode)
        """
            if event.type == pygame.KEYUP:
                if event.key == pygame.K_LEFT:
                    left = False
                if event.key == pygame.K_RIGHT:
                    right = False
        """
        if action == 0:
            left = True
        elif action == 1:
            left = right = False
        elif action == 2:
            right = True

        if left:
            self.player.moveLeft()
            left = False
        if right:
            self.player.moveRight()
            right = False

        stateDash = self.getState()

        self.gameDisplay.fill((23, 24, 25))
        self.bricks.displayBricks(self.gameDisplay)
        self.player.drawPaddle(self.gameDisplay)
        self.ball.drawBall(self.gameDisplay)
        self.sayMessage('Score: ' + str(self.score))
        self.sayMessage('M. Score: ' + str(self.maxScore), loc=((self.windowWidth // 2 - 120, 0)))
        self.sayMessage('BREAKOUT', color=(200, 150, 230), loc=(self.windowWidth // 2 - 30, 0))
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
        self.sayMessage('Episode ' + str(episode), color=(200, 150, 230), loc=(self.windowWidth -
100, 0))
        #print(self.player.Qtable)
        self.ballMove()


        if (self.player.x <= self.ball.x <= self.player.x + self.player.paddleWidth) and
(self.ball.y + self.ball.rad >= self.player.y):
            reward += 5
        reward += self.checkDead()
        self.player.learn(state, action, reward, stateDash)
        state = stateDash
        """
        # Check if ball is out of bounds, then Game Over
         if self.player.dead:
            self.sayMessage('GAME OVER', loc=(self.windowWidth // 2 - 50, self.windowHeight // 2))
            pygame.display.update()
            time.sleep(2)
        """
        if len(self.bricks.cordBricks) == 0:
            self.sayMessage('YOU WON!!!', loc=(self.windowWidth // 2 - 50, self.windowHeight // 2))
            pygame.display.update()
            time.sleep(2)
            pygame.quit()


        pygame.display.update()
        window = pygame.surfarray.array2d(self.gameDisplay)
      return window


#######DRL AGENT
def apply_gradients(gradients, model_parameters, learning_rate):
   for i in range(len(model_parameters)):
      model_parameters[i] -= learning_rate * gradients[i]
def huber_loss(target, prediction, delta=1.0):
   error = prediction - target
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
    is_small_error = np.abs(error) < delta
    squared_loss = 0.5 * np.square(error)
    linear_loss = delta * (np.abs(error) - 0.5 * delta)
    return np.where(is_small_error, squared_loss, linear_loss)
def one_hot(indices, depth):
    one_hot_encoded = [[0 if i != index else 1 for i in range(depth)] for index in indices]
    return one_hot_encoded
def DeepQ(self, episode, model, target, optimiser, left = False, right = False,
    number_actions = 4,
    gamma = 0.99,
    epsilon = 1.0,
    batch_size = 32,
    max_step = 1000,
    max_memory = 100000,
    update_action_num = 4,
    update_network = 1000,
    First = True,
    window = np.array([[0,0],[0,0]]),
    episode_reward = 0,
    loss_function = huber_loss(),
    history = {'Action': [], 'State': [],
               'State_next': [],
               'Reward': [],
               'Finished': [],
               'Episode_reward': []}):

    while not self.player.dead:
        self.gameDisplay.fill((23, 24, 25))
        self.bricks.displayBricks(self.gameDisplay)
        self.player.drawPaddle(self.gameDisplay)
        self.ball.drawBall(self.gameDisplay)
        self.sayMessage('Score: ' + str(self.score))
        self.sayMessage('M. Score: ' + str(self.maxScore), loc=((self.windowWidth // 2 - 120, 0)))
        self.sayMessage('BREAKOUT', color=(200, 150, 230), loc=(self.windowWidth // 2 - 30, 0))
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
        self.sayMessage('Episode ' + str(episode), color=(200, 150, 230), loc=(self.windowWidth -
100, 0))
        self.ballMove()
        action = select_action(epsilon, window, history['State'])
        epsilon = probability_decay(epsilon)
        reward = 0
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                return
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_LEFT:
                    left = True
                if event.key == pygame.K_RIGHT:
                    right = True


        next_window = pygame.surfarray.array2d(self.gameDisplay)
        if not First:
            history = update_history(action, preprocess(window), preprocess(next_window), reward,
finished, Dict = history)
            #LEARN
            frame_count = len(history['Finished'])
            if frame_count % update_action_num == 0 and frame_count > batch_size:
                ### Get indexes
                index = np.random.choice(range(len(history['Finished'])), size=batch_size)



                ### Sample features by using the indexes


                sample_Action = feature_sampling('Action', history, index)
                sample_State = feature_sampling('State', history, index)
                sample_State_next = feature_sampling('State_next', history, index)
                sample_Reward = feature_sampling('Reward', history, index)
                sample_Finished = [float(history['Finished'][i]) for i in index]
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
            ### Find Q values for feature rewards
            max_future_rewards = [max(reward) for reward in reward_future]
            reward_future = target.predict(sample_State_next)
            updated_q_values = [sample_Reward[i] + gamma * max_future_rewards[i] for i in
range(len(sample_Reward))]
            updated_q_values = updated_q_values * (1 - sample_Finished) - sample_Finished

            ### Create mask
            masks = one_hot(sample_Action, number_actions)

            ### Train the model
            q_values = [model(state) for state in sample_State]
            ### Apply the masks to the Q-values to get the Q-value for action taken
            q_action = [sum(q * m for q, m in zip(q_value, mask)) for q_value, mask in zip(q_values,
masks)]
            ### Find loss between new and old q values
            loss = loss_function(updated_q_values, q_action)

            ### Backpropagation
            grads = grads = backpropagation(loss, model_parameters)
            apply_gradients(grads, model_parameters, learn_rate)

        if frame_count % update_network == 0:
            ### Update the network
            target.set_weights(model.get_weights())

            window = next_window
        else:
            window = next_window
            First = False

        if len(history['Reward']) > max_memory:
         history = del_history(history)
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
            if action == 0:
                left = True

            elif action == 1:
                left = right = False
            elif action == 2:
                right = True

            if left:
                self.player.moveLeft()
                left = False
            if right:
                self.player.moveRight()
                right = False



            if (self.player.x <= self.ball.x <= self.player.x + self.player.paddleWidth) and (self.ball.y +
self.ball.rad >= self.player.y):
                reward += 5
                reward += self.checkDead()
                finished =  len(self.bricks.cordBricks) == 0

            if finished:
             break
            if len(self.bricks.cordBricks) == 0:
                self.sayMessage('YOU WON!!!', loc=(self.windowWidth // 2 - 50, self.windowHeight //
2))
            pygame.display.update()

            time.sleep(2)
            pygame.quit()

            pygame.display.update()
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120

```
            episode_reward += reward

        history['Episode_reward'].append(episode_reward)
        log(episode, history['Episode_reward'][-1])
        return model, target, optimiser, history, epsilon

model, target, optimiser = network_model(number_actions = 4,
                                         learn_rate = 0.00025)

game = Game()
hist = {'Action': [], 'State': [], 'State_next': [],'Reward': [],'Finished': [],'Episode_reward': []}
epsilon = 1.0
log(1,1, first = True)

for episode in range(1, 1000):
    model, target, optimiser, hist, epsilon = game.DeepQ(episode, model, target, optimiser, history =
hist, epsilon = epsilon)
    game.reset()
pygame.quit()
```

Muhammed Gölcük 21802551

Metehan Küçük 21802120